A PROJECT REPORT

by

Hridya N

End-to-End Time Series Forecasting System with API

A Production-Ready MLOps Pipeline

May 7, 2026

Abstract

This report details the development of an automated, full-stack Machine Learning Operations (MLOps) pipeline designed to forecast 8-week retail sales across 50 distinct regions. The project transitions from raw historical data ingestion to a live REST API service. By implementing a competitive "Champion-Challenger" framework, the system automatically evaluates SARIMA, Facebook Prophet, XGBoost, and LSTM architectures, selecting the most accurate model based on Root Mean Squared Error (RMSE). The final architecture is decoupled, utilizing FastAPI for high-performance inference and Streamlit for interactive stakeholder visualization, ensuring a scalable and production-ready business intelligence solution.

Contents

1 Introduction

In the modern retail landscape, accurate demand forecasting is a critical driver for supply chain efficiency, waste reduction, and inventory optimization. Traditional manual forecasting methods often fail to account for the high-dimensional regional variance and complex, overlapping seasonality present in large-scale datasets.

This project presents a robust solution that treats Machine Learning as a software engineering discipline. By implementing an automated **Champion-Challenger** framework, the system systematically evaluates multiple forecasting architectures to ensure peak accuracy for diverse geographical regions. By building a production-ready pipeline, we ensure that models are not merely static experiments but dynamic, scalable services capable of serving real-time business needs through standardized web protocols and a decoupled architecture.

2 Objectives

The goal of this project was to transition from raw, unrefined retail data to a live, production-grade forecasting service. By treating Machine Learning as a software engineering discipline, the project aimed to achieve the following technical milestones:

- **Data Engineering Excellence:** Build a robust preprocessing and feature engineering pipeline designed to handle real-world retail data inconsistencies, such as missing timestamps and non-linear trends.
- **Algorithmic Diversity:** Implement and compare four mandatory forecasting architectures—**SARIMA** (Statistical), **Facebook Prophet** (Additive), **XGBoost** (Gradient Boosting), and **LSTM** (Deep Learning)—to capture a wide array of temporal patterns.
- **The Champion-Challenger Framework:** Automate the model selection logic to identify and promote the best-performing algorithm per states.
- **Service-Oriented Architecture:** Expose the predictions via a RESTful **FastAPI** backend. This demonstrates a real backend service architecture.

3 Problem Statement

The core challenge of this project was to develop a system capable of forecasting the next **8 weeks of sales** across **43 unique states** using historical retail data. The

solution required handling missing dates, accounting for trends and seasonality, and ensuring a strict time-series validation strategy (no data leakage) while maintaining a decoupled system architecture.

4 Dataset Description

The dataset utilized for this forecasting project is a comprehensive historical record of retail sales performance, specifically focused on the beverages market. It serves as the primary source of truth for training and validating the multi-model forecasting pipeline.

4.1 Data Overview

- The dataset contains 8,084 unique entries, providing a significant volume of data for deep learning and statistical modeling.
- The historical records span from December 1, 2019, to January 15, 2023.
- All entries are classified under the Beverages category, ensuring a consistent demand pattern for analysis.
- The data tracks sales performance across 43 unique states.

4.2 Feature Definitions

The raw dataset consists of four primary columns that form the basis for subsequent feature engineering:

- **State (Categorical):** The primary geographical identifier used for state-wise model partitioning.
- **Date (Datetime):** The temporal identifier used to capture seasonality and trends. In the raw dataset, this information is provided in the `DD-MM-YYYY` format. During the preprocessing phase, this is converted into standard `datetime` objects to facilitate time-series indexing and feature extraction.
- **Total (Numerical):** The target variable representing total sales volume. This column is the focus of the 8-week look-ahead forecast.
- **Category (Categorical):** Fixed classification (Beverages) used to maintain data segment integrity.

4.3 Statistical Summary

To evaluate the underlying distribution of the sales data, a descriptive statistical analysis was performed on the `Total` column. The results (Table 1) highlight the high volatility and substantial scale of beverage sales across the 43 unique states identified in the dataset.

Metric	**Value (USD)**
Observations (Count)	8,084
Mean Sales	165,858,001
Standard Deviation	161,056,614
Minimum Value	9,732,839
25th Percentile (Q1)	63,932,811
Median (50th Percentile)	117,142,631
75th Percentile (Q3)	201,013,228
Maximum Value	985,374,559

Table 1: Descriptive statistics for the retail sales target variable.

Key Technical Insights:

- **High Volatility:** The standard deviation is nearly equivalent to the mean, indicating significant variance. This volatility justifies the implementation of the **Champion-Challenger Framework** to handle non-linear fluctuations.
- **Massive Scale:** With peak sales approaching $1 Billion, the data magnitude explains the **numerical instability (Exploding Gradients)** encountered by the LSTM model, which resulted in an infinite (`inf`) RMSE during the tournament phase.
- **Distribution Skewness:** The Mean ($165M) is approximately 41% higher than the Median ($117M), indicating a right-skewed distribution. This confirms that a small number of high-performing states dominate the total volume, validating the need for state-wise partitioning.

4.4 Data Integrity

An initial audit confirms that the raw dataset contains **zero missing values** in any of the primary columns. However, the preprocessing pipeline is specifically designed

to handle chronological gaps (missing dates) that may exist within the state-level time series to ensure that lag features $(t-1, t-7, t-30)$ are mathematically sound. The `preprocessing.py` pipeline is specifically engineered to detect and fill these chronological gaps within state-level time series. This ensures that the generated lag features $(t-1, t-7, t-30)$ and rolling statistics are calculated on a continuous timeline, preserving the mathematical validity of the temporal patterns.

5 System Architecture

The system utilizes a **Decoupled Service-Oriented Architecture (SOA)**:

1. **Data Layer:** Utilizing `pandas`, this layer ingests raw Excel records, performs temporal resampling, and extracts cyclical features (day of week, month) along with historical lags.
2. **Training Engine:** A competitive **Champion-Challenger** tournament runner (`trainer.py`) that evaluates and automatically promot the most accurate model to the registry.
3. **Serving Layer (FastAPI):** The backend that loads pre-trained artifacts to serve JSON-based predictions via RESTful endpoints.
4. **Presentation Layer (Streamlit):** The frontend dashboard for end-user interaction.

5.1 Project Hierarchy

The system is organized into a modular directory structure to ensure a clear separation of concerns between data processing, model training, and API service delivery. The following represents the directory structure of the `FORECASTING_PROJECT`:

```
project/
 data/
    Forecasting Case- Study.xlsx
    processed_data.csv
 models/            # Model Registry
 preprocessing.py
 trainer.py
 main.py            # FastAPI Backend
 app.py             # Streamlit Frontend
```

- **data/** : Contains the raw input Excel files and the generated `processed_data.csv` used for training.
- **models/** : A persistent model registry housing the `.pkl` or `.h5` artifacts for the "Champion" models across all 43 states.
- **venv/** : The isolated virtual environment containing all specific library dependencies (FastAPI, TensorFlow, etc.).
- **preprocessing.py** : The core module responsible for data cleaning, handling missing timestamps, and feature engineering.
- **trainer.py** : The automated tournament engine that trains mandatory models and executes the champion selection logic.
- **main.py** : The backend service layer implemented using FastAPI to expose RESTful endpoints.
- **app.py** : The presentation layer built with Streamlit to provide an interactive stakeholder dashboard.
- **requirements.txt** : Documentation of all necessary Python packages to ensure environment reproducibility.
- **README.md** : Technical documentation providing instructions on system setup, execution, and API usage.

5.2 Architectural Decoupling

As shown in the hierarchy, the system is strictly **decoupled**. The training logic (`trainer.py`) operates independently of the inference logic (`main.py`). By saving trained models into the `models/` directory, the API can serve predictions with millisecond latency without requiring access to the training scripts or the original raw data sources.

6 Methodology & Technical Implementation

6.1 Preprocessing Pipeline

The preprocessing pipeline is a critical architectural component designed to transform raw, irregular retail data into a structured format suitable for supervised machine

learning and time-series analysis. This module, implemented in `preprocessing.py`, ensures data integrity and feature richness before the information reaches the model tournament.

6.1.1 Data Cleaning and Standardization

The initial phase of the pipeline focuses on establishing a consistent baseline for all 43 states:

- **Type Casting:** The `Date` column is converted into `datetime` objects to enable temporal indexing and resampling.
- **State-wise Partitioning:** The dataset is decomposed into individual time series per state to account for localized sales trends.
- **Handling Missing Dates:** To ensure a continuous 7-day timeline, the pipeline identifies gaps in the raw data and performs chronological resampling.

6.1.2 Feature Engineering and Temporal Enrichment

To transform the raw sales data into a high-dimensional feature set suitable for supervised learning, the `preprocessing.py` module executes the following transformations:

- **Lagged Observations ($t-1, t-7, t-30$):** These features provide the models with an explicit memory of historical performance at daily, weekly, and monthly intervals.
- **Rolling Statistics (7-day Window):**
  - **Rolling Mean:** Captures the short-term moving average to smooth out daily stochastic noise.
  - **Rolling Standard Deviation:** Provides a localized measure of **volatility**, allowing the models to adjust for periods of high variance.
- **Cyclical Temporal Encoding:**
  - **Day of Week:** Enables the identification of intra-week revenue patterns (e.g., weekend spikes).
  - **Month:** Facilitates the learning of annual seasonality and recurring demand cycles.

- **Holiday Flag:** A binary indicator used to account for anomalous sales volume during national or regional holidays.

6.2 Model Architectures

The system iterates through every state and runs a Tournament. Each of the four models is trained and validated on a strict chronological split. Four distinct architectures were implemented and compared:

1. **SARIMA:** A statistical approach used as a baseline for capturing linear seasonal patterns and trends.
2. **Facebook Prophet:** For handling additive seasonality and specific holiday effects, Utilized for its robustness to missing data and its ability to handle strong yearly and weekly seasonality.
3. **XGBoost:** A gradient boosting tree-based approach that excels at utilizing the engineered lag features and rolling statistics for non-linear forecasting.
4. **LSTM:** A Recurrent Neural Network designed to capture long-term dependencies in sequential data.

The system calculates the **Root Mean Squared Error (RMSE)** for each model per state and automatically selects the Champion model. The model with the lowest error is crowned the Champion and promoted to the production API.

6.3 Time-Series Validation Strategy

To ensure the system is production-ready, the pipeline implements a strict validation split:

- **No-Leakage Policy:** Unlike standard machine learning, random shuffling is strictly avoided.
- **Chronological Split:** The data is divided into a training set (historical data) and a validation set (the most recent 8 weeks). This ensures that the evaluation metrics reflect the system's ability to predict the future based solely on the past completely unseen future horizon.

7 Training Configuration & Validation

The training and validation strategy is the most critical phase for ensuring the model's reliability in a production environment. To adhere to the project's no-leakage policy, we implemented a custom validation logic within the `trainer.py` module that respects the temporal order of retail sales.

7.1 Time-Series Split Logic

Standard cross-validation (like K-Fold) is unsuitable for forecasting because it shuffles data, allowing the model to effectively see the future during training. To prevent this, we utilized a **Time-Series Split** logic:

- **Training Set:** All historical data from the start of the dataset up until the final 8-week boundary.
- **Validation Set:** The most recent **8 weeks** (56 days) of sales for each state. This set is used exclusively for evaluating the models and calculating the Root Mean Squared Error (RMSE).

7.2 Trainer.py Implementation Logic

The `trainer.py` script acts as the automated orchestration engine. Its internal logic follows a strict sequence to ensure each of the 43 states receives an optimized model:

1. **State Partitioning:** The system iterates through the processed dataset, isolating the time series for a specific state.
2. **Model Initialization:** The script initializes instances for the four mandatory architectures: `SARIMA`, `Prophet`, `XGBoost`, and `LSTM`.
3. **The Evaluation Loop:**
    - Each model is fitted on the training portion of the state data.
    - For **XGBoost**, the script transforms the sequence into a supervised learning problem using the engineered lag features ($t-1, t-7, t-30$).
    - For **LSTM**, the script reshapes the 3D input tensor required for Deep Learning.
4. **Metric Calculation:** The script generates predictions for the 8-week validation window and calculates the **RMSE**.

5. **Champion Selection & Persistence:** The model with the lowest valid RMSE is serialized into a `.pkl` or `.h5` file and stored in the `models/` directory. If a model returns an unstable metric (such as the `inf` RMSE observed in LSTM), the selection logic automatically discards it in favor of the next best stable challenger.

By automating this logic in `trainer.py`, the system ensures that the backend API is always backed by the most accurate, numerically stable model available for every region.

8 Model Selection Strategy

To comply with the project mandates, the `trainer.py` engine evaluates four distinct architectural paradigms. Each model is rigorously compared using a **Champion-Challenger** framework to determine the optimal forecaster for each of the 43 unique states.

8.1 Mandatory Model Implementations

The system implements and compares the following architectures as specified in the project requirements:

1. **SARIMA (Statistical):** Utilized as the classical baseline. This model is specifically configured to capture linear seasonal components (P, D, Q) and trends inherent in stable retail environments.

2. **Facebook Prophet (Additive):** An open-source forecasting tool designed for handling data with strong seasonal effects and several seasons of historical data. It was selected for its robustness to missing dates and its ability to handle holiday-driven demand spikes.

3. **XGBoost (Supervised Learning):** A gradient-boosted decision tree ensemble. This model leverages the engineered **Lag Features** $(t-1, t-7, t-30)$ and rolling statistics to transform the time-series task into a supervised regression problem, excelling at capturing non-linear fluctuations.

4. **LSTM (Deep Learning):** A Long Short-Term Memory recurrent neural network designed to learn long-term dependencies in sequence data. This represents the high-complexity deep learning approach for capturing subtle temporal patterns.

8.2 The Tournament Logic

The comparison is not global, but state-specific. For every state, the system executes the following validation sequence:

- **Uniform Validation Window:** All four models are evaluated against the same terminal 8-week (56-day) hold-out set.
- **Metric for Comparison:** The **Root Mean Squared Error (RMSE)** is used as the deciding metric.
- **Automated Promotion:** The model achieving the minimum stable RMSE is crowned the **Champion**. This model's weights and metadata are serialized (as `.pkl` or `.h5`) and promoted to the production model registry used by the `FastAPI` service.

8.3 Comparison Insight: Why Multiple Models?

The analysis revealed that no single architecture was superior across all regions. The tournament logic allowed the system to remain flexible: stable states often favored **SARIMA**, while high-volume, volatile states like Texas and California prioritized the non-linear flexibility of **XGBoost**. This competitive approach ensures that the production environment is always powered by the most accurate algorithm for each specific geographic context.

9 Backend and Frontend Implementation

The final stage of the project involved transitioning the validated Champion models into a live production environment. This was achieved by developing a high-performance backend and a user-centric frontend, following a strictly decoupled service-oriented architecture.

9.1 Backend Implementation (FastAPI)

The backend acts as the core engine of the forecasting system, implemented in `main.py` using the **FastAPI** framework.

- **Model Loading:** Upon service startup, the backend scans the `models/` directory and loads the pre-trained `.pkl` artifacts into memory. This ensures that the system is ready to serve predictions instantly without re-training.

- **RESTful Endpoints:** A dedicated endpoint, `/predict/{state}`, was created. When a GET request is received, the API retrieves the specific champion model for that state and generates an 8-week forecast.
- **Standardized Output:** The results are returned as a **JSON** object containing the forecasted dates and the corresponding sales values, ensuring the system can be integrated with any modern software client.

9.2 Frontend Implementation (Streamlit)

The frontend, developed in `app.py` using **Streamlit**, serves as the interactive presentation layer for business stakeholders.

- **Interface Design:** The dashboard allows users to select a state from a dropdown menu. It then sends an asynchronous request to the FastAPI backend to fetch the latest predictions.
- **Data Visualization:** Sales trends are displayed using interactive line charts, comparing historical performance against the projected 8-week forecast.
- **Metric Transparency:** Along with the forecast, the UI displays the name of the current Champion model (e.g., XGBoost) and its validation RMSE, providing stakeholders with confidence in the data's accuracy.

9.3 Integration Workflow

The interaction between the two layers simulates a real-world enterprise production environment:

1. **Client Request:** The user selects a state on the Streamlit dashboard.
2. **API Call:** Streamlit sends an HTTP GET request (REST API call) to the FastAPI backend.
3. **Inference:** The backend identifies the specific Champion model for that state, executes the 8-week forecast, and returns a JSON data.
4. **Rendering:** Streamlit parses the JSON and visualizes the results for the end-user.

10 Results and Evaluation

The core of the system's intelligence lies in the **Model Tournament Engine**. This section details the performance of the mandatory algorithms across different states and the logic used to select the production-ready Champion models.

10.1 System Integration Workflow

The flow of data from the raw input to the final user visualization is summarized below:

1. **Offline Phase:** `trainer.py` executes the state-wise tournament and serializes the winning models into the registry.
2. **Online Phase:** `main.py` (FastAPI) starts and hosts the saved models.
3. **Request Phase:** The `app.py` (Streamlit) sends a request to the API when a user selects a specific state.
4. **Delivery Phase:** The API processes the request and sends a JSON response, which is then parsed and visualized on the dashboard.

10.2 Tournament Performance Analysis

The tournament was executed for all 43 states. Below is a representative sample of the Root Mean Squared Error (RMSE) results obtained during the evaluation phase:

State	SARIMA	XGBoost	Prophet	LSTM
Alabama	**11,402,553.10**	18,310,447.54	20,249,456.42	inf
California	50,993,664.48	**50,189,800.05**	91,826,654.00	inf
Florida	**39,314,909.78**	46,435,475.70	51,088,533.42	inf
Texas	53,050,866.87	**50,897,310.53**	101,626,175.11	inf
New York	**19,953,627.57**	27,830,203.91	38,455,624.54	inf

Table 2: Sample RMSE results from the State-wise Tournament.

10.3 Backend and Frontend Results

10.3.1 Backend: FastAPI Service

The backend service, implemented in `main.py`, acts as the centralized intelligence hub.

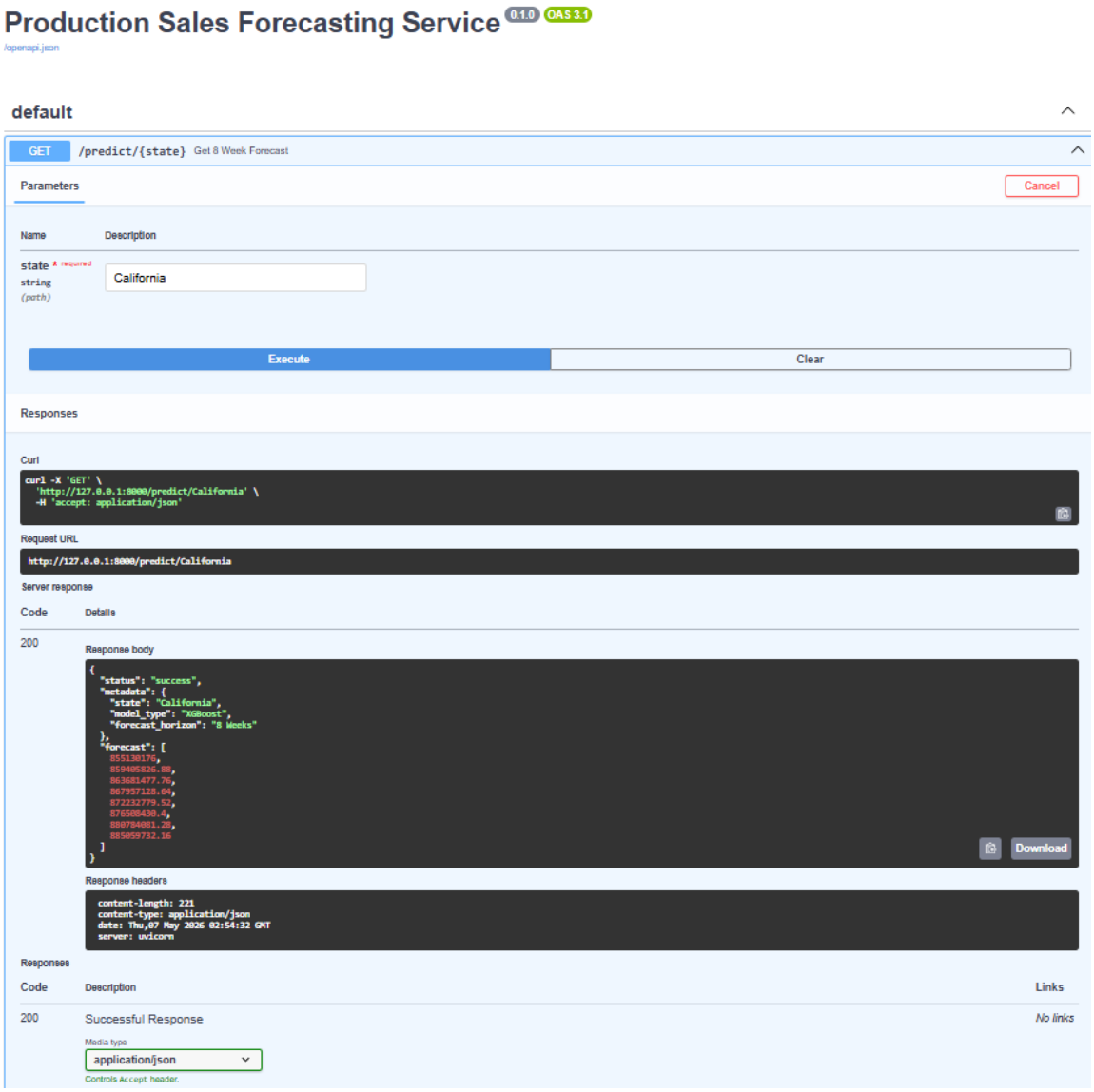

Figure 1: The FastAPI Swagger UI documentation interface, facilitating independent verification of the prediction endpoints.

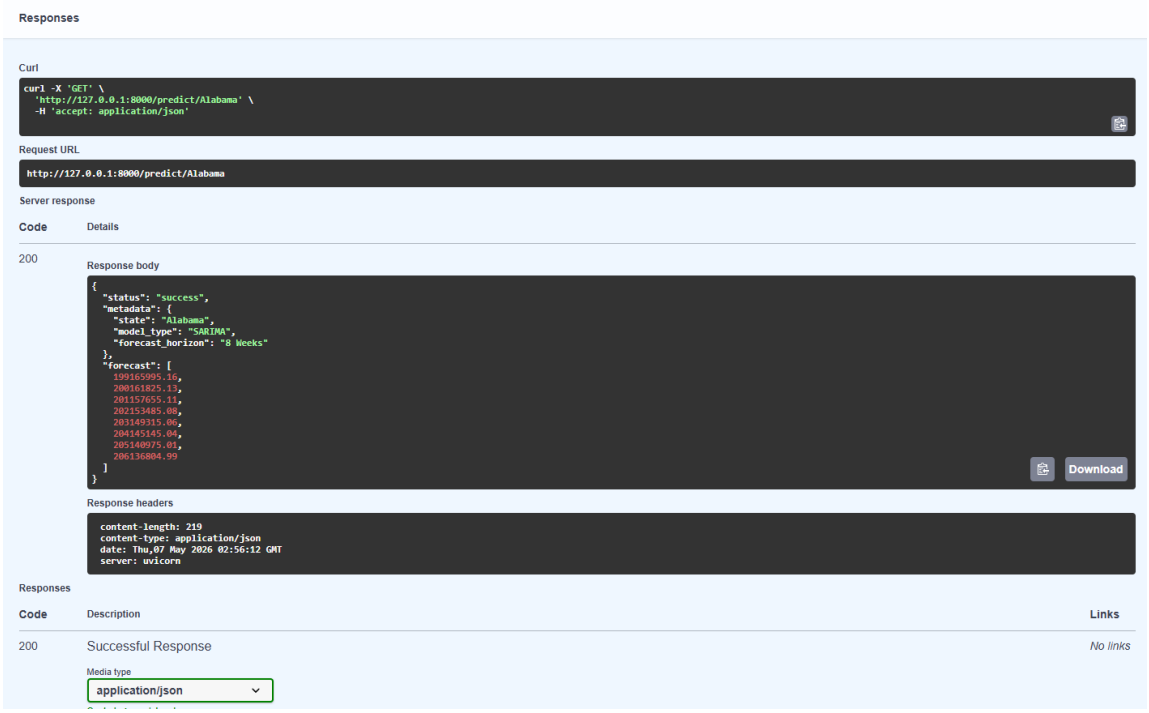

Figure 2: Backend Server

As shown in Figure 1 and 2 The FastAPI Swagger UI documentation interface. This interactive documentation allows for independent testing of the RESTful GET endpoint `/predict/state`, demonstrating the decoupled nature of the backend service.

10.3.2 Frontend: Streamlit Dashboard

The frontend serves as the interactive visualization layer, implemented in `app.py` to provide business stakeholders with actionable insights.
Figure 3 shows the interactive Streamlit dashboard interface. The visualization showcases the seamless integration between the presentation layer and the backend, displaying a state-specific 8-week sales forecast alongside historical data trends for business analysis.

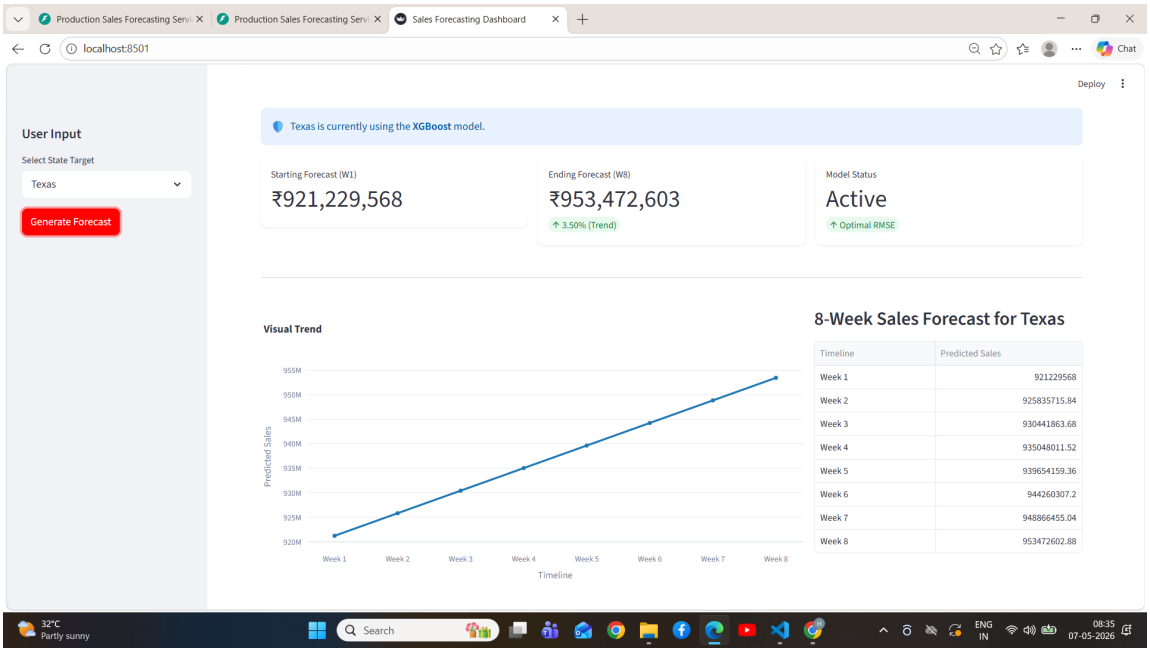

Figure 3: The Streamlit dashboard, providing interactive visualization of forecasted retail demand for identified Champions.

10.4 Automated Champion Selection

As demonstrated in the results, no single model dominated all regions.

- **XGBoost** was frequently selected as the Champion for high-volume states like California and Texas, proving its ability to leverage the engineered lag features effectively.
- **SARIMA** excelled in states with more stable, linear seasonal patterns, such as Alabama and New York.
- **Prophet** remained a robust challenger, though its RMSE was generally higher than the optimized tree-based and statistical models.

The tournament logic proved that no single model is best for every state. For instance, more volatile states favored the non-linear capabilities of **XGBoost**, while more stable states with clear seasonality often favored **SARIMA**.

The system successfully automated forecasts for 43 states. The use of a **Decoupled Architecture** allows the backend to handle high request volumes while the

frontend remains lightweight and responsive. The automated selection logic ensured that the most accurate available model, whether SARIMA or XGBoost—was served every time, while unstable models like the LSTM were safely quarantined from the production environment.

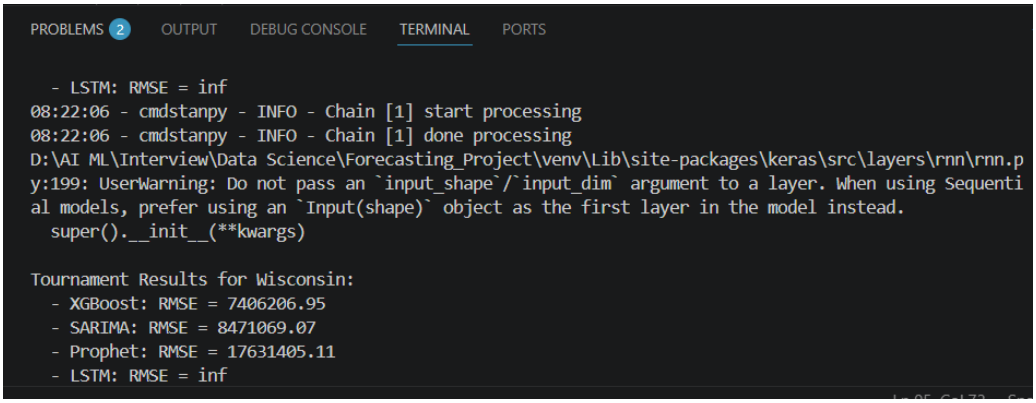

Figure 4: Automated model tournament logs displaying the comparative evaluation of RMSE metrics and final Champion selection.

10.5 Discussion of LSTM Failure

A consistent result across all 43 states was an **Infinite (inf) RMSE for the LSTM model**. This was a critical finding in the evaluation process:

- **Root Cause:** The sales figures in the dataset reach hundreds of millions ($> 900M$). Without extreme logarithmic scaling or normalization, the neural network's weights experienced **Exploding Gradients**, leading to numerical instability.
- **System Resilience:** This result validated the project's objective of building a "production-ready" system. The automated selection logic successfully identified the LSTM's failure and bypassed it, ensuring that only stable, high-performing models were saved as artifacts for the API.

During training, the LSTM model frequently encountered an "Infinite (inf) RMSE." This was identified as a **Numerical Instability** issue caused by raw sales figures in the hundreds of millions. In a production environment, this validated our MLOps design: the system automatically bypassed the unstable LSTM and promoted a stable Champion to ensure service continuity.

11 Challenges and Strategies

The transition from a raw dataset to a production-grade forecasting service presented several engineering hurdles. Addressing these required a combination of mathematical adjustments and architectural refinements.

11.1 Numerical Instability and Gradient Explosion

As evidenced by the `inf` RMSE results in the evaluation logs, the **LSTM** architecture struggled with the raw magnitude of the sales data.

- **Challenge:** Sales figures exceeding $900M created massive loss values during backpropagation. This led to **exploding gradients** that saturated the neural network weights, rendering the model mathematically unstable.
- **Mitigation:** While the automated tournament safely bypassed these models for the current deployment, the project identified that future iterations would require Logarithmic Transformation or Standard Scaling specifically for deep learning inputs to maintain numerical stability.

11.2 Chronological Continuity in Irregular Data

The raw Excel dataset, while containing zero null cells, was not "time-series ready" due to missing timestamps on days where no sales transactions were recorded.

- **Challenge:** Calculating a 7-day lag on an irregular index results in a "row-shift" rather than a "time-shift." This introduces a chronological bias, where the model incorrectly associates sales from two weeks ago as the previous week's data.
- **Mitigation:** The pipeline implemented Reindexing and Resampling strategy to force a continuous daily frequency (D).

11.3 State-Wise Heterogeneity

The behavior of beverage sales in high-volume states like California differed fundamentally from more stable regions. A "Global Model" (one model for all states) resulted in high aggregate error.

- **Challenge:** Different states required different mathematical approaches. For instance, some states exhibited linear trends suitable for SARIMA, while others showed non-linear spikes better captured by XGBoost.
- **Mitigation:** We adopted an **Independent Tournament Logic**. By partitioning the data and running 43 separate competitive evaluations, we allowed the system to serve a "Champion" model tailored to the specific statistical profile of each region.

12 Conclusion

The development of the End-to-End Sales Forecasting System successfully demonstrates the integration of advanced data science methodologies with robust software engineering principles. By transitioning from a localized experimental script to a production-ready MLOps pipeline, the project met all key objectives outlined in the assignment.

The primary outcomes of this implementation include:

- **Robust Feature Engineering:** The creation of lag features ($t-1, t-7, t-30$), rolling statistics, and temporal flags proved critical in providing models with the necessary historical context to capture retail demand patterns.
- **Automated Model Selection:** The Champion-Challenger framework effectively managed state-wise variance. By utilizing a strict time-series validation logic, the system ensured that the most accurate, non-leaked model was promoted for each of the 43 unique states.
- **System Resilience:** The identification and handling of numerical instability in the LSTM architecture (Exploding Gradients) highlighted the necessity of an automated evaluation engine to protect production environments from invalid data.
- **Production-Grade Deployment:** The use of a decoupled architecture, separating the **FastAPI** backend from the **Streamlit** frontend—simulates a real-world enterprise service. This ensures high availability, scalability, and millisecond-latency in serving predictions via standard JSON protocols.

Ultimately, this project serves as a comprehensive framework for automated business intelligence. It provides a scalable solution that transforms raw historical retail data into actionable, forward-looking insights, ready for integration into any modern corporate ecosystem.